

FoodieConnect

Location-Based Food Delivery Application

Project Report

A MERN Stack Web Application

Developed By

Swati Muttannavar

Computer Science and Engineering

Academic Project Report

1. Abstract

FoodieConnect is a full-stack, location-based food delivery web application developed using the MERN stack. It connects customers with restaurants that can serve their selected delivery location. Customers can register, enter a normal delivery address, browse restaurants and menus, add food items to a cart, and place orders. Restaurant administrators can manage menus and incoming orders, while delivery agents can manage assigned deliveries.

A key feature is address-based location detection. Users do not manually enter latitude and longitude. A map or geocoding service converts the entered address into geographical coordinates in the background, which can then be used for restaurant matching.

2. Introduction

Food delivery platforms require coordination between customers, restaurants, and delivery personnel. FoodieConnect addresses this requirement by combining food ordering with location-aware restaurant discovery. The project demonstrates React frontend development, Node/Express REST APIs, MongoDB database management, authentication, role-based access, cart management, order processing, and geospatial location handling.

3. Problem Statement

Customers need a simple way to discover restaurants that can deliver to their selected location. Restaurant administrators need tools to manage menus and orders, while delivery agents need access to assigned deliveries. FoodieConnect provides a centralized system for coordinating these activities.

4. Objectives

- Develop a complete food delivery platform using the MERN stack.
- Allow customers to register and securely log in.
- Identify delivery locations automatically from entered addresses.
- Display restaurants that are serviceable for the customer's location.
- Provide menu browsing and cart functionality.
- Allow customers to place and track orders.
- Allow restaurant administrators to manage food items and orders.
- Provide delivery agents with assigned order information.
- Implement secure authentication and role-based authorization.

5. Scope of the Project

FoodieConnect covers the workflow from user registration and location selection to restaurant discovery, ordering, restaurant processing, delivery assignment, and order completion. It is designed to support multiple restaurants in different towns or areas, with each restaurant maintaining its own account and operational data.

6. System Users and Roles

Role	Main Responsibilities
Customer	Register, login, select delivery address, browse restaurants, manage cart, place orders, and view status.
Restaurant Admin	Register restaurant, manage menu items, receive orders, and update preparation status.
Delivery Agent	View assigned orders and update pickup, delivery, and completion status.
Super Admin	Manage users, restaurants, delivery agents, and overall platform activity.

7. Technology Stack

Frontend: React.js, JavaScript, HTML5, CSS3, Tailwind CSS, Axios, React Router.

Backend: Node.js, Express.js, REST APIs, JWT, bcrypt, dotenv, CORS.

Database: MongoDB, Mongoose, MongoDB Atlas.

Location Services: Map/geocoding API and geographical coordinate-based matching.

8. System Architecture

FoodieConnect follows a client-server architecture. The React frontend provides the interface. Node.js and Express.js handle authentication, business logic, validation, and REST APIs. MongoDB stores users, restaurants, food items, addresses, orders, and delivery information.

React Frontend → Express/Node.js REST API → Mongoose → MongoDB

9. Location-Based Restaurant Matching

During registration, a customer enters an ordinary address such as house/building, area, city/town, and pincode. A map/geocoding service identifies the location and returns coordinates automatically. The coordinates are stored in the backend without requiring the customer to type them. Restaurant registration follows the same approach. The application can then use geographical queries and a configured delivery radius or service area to identify serviceable restaurants.

10. Registration and Login

Customer registration includes name, email, phone number, password, confirm password, and delivery address. Restaurant registration includes restaurant name, owner name, contact details, password, and restaurant address. Users log in using email and password. The backend identifies the user's role and provides access to the appropriate dashboard.

11. Food Ordering Workflow

- Customer registers or logs in.
- Customer enters and confirms a delivery address.
- The application identifies the location automatically.
- Serviceable restaurants are displayed.
- Customer selects a restaurant and browses its menu.
- Food items are added to the cart.

- Customer reviews the cart and confirms the delivery address.
- Order is placed and stored in MongoDB.
- The selected restaurant receives the order.
- Restaurant accepts and prepares the order.
- A delivery agent is assigned.
- The delivery agent picks up and delivers the order.
- The order is marked as delivered.

12. Cart Management

The cart allows customers to add food items, change quantities, remove items, and calculate the total amount. A single-restaurant cart can be enforced so that one order contains items from one restaurant, simplifying order routing.

13. Order Management

An order stores the customer, restaurant, ordered items, quantities, prices, total amount, delivery address, location, payment status, order status, delivery agent, and timestamps. A typical status flow is: Pending → Accepted → Preparing → Ready for Pickup → Picked Up → Out for Delivery → Delivered.

14. Database Design

The proposed MongoDB collections include Users, Restaurants, FoodItems, Orders, DeliveryAgents, and Addresses. Restaurant documents contain address and location information. Order documents reference the customer, restaurant, ordered items, and delivery details.

15. Security

- Passwords are protected using bcrypt hashing.
- JWT can be used for authenticated sessions.
- Protected routes restrict access to authorized users.
- Role-based authorization separates customer, restaurant, delivery, and admin operations.
- Environment variables protect sensitive configuration such as database and API keys.
- Input validation should be applied to registration, login, menu, and order APIs.

16. Project Structure

FoodieConnect/

■■■ frontend/

■ ■■■■ src/

■ ■■■■ public/

■ ■■■■ package.json

■■■ backend/

■ ■■■■ controllers/

■ ■■■■ models/

- ■■■ routes/
- ■■■ middleware/
- ■■■ package.json
- reports/
- README.md
- .gitignore

17. Testing

- Registration and login validation.
- Restaurant and food item CRUD operations.
- Address and location detection.
- Restaurant filtering based on delivery location.
- Cart quantity and total calculations.
- Order placement and status updates.
- Role-based access control.
- Frontend-backend API integration.
- Database connectivity and error handling.
- Responsive user interface testing.

18. Advantages

- Simple address-based location selection.
- Supports multiple restaurants and locations.
- Separate dashboards for different roles.
- Centralized order management.
- Scalable MERN architecture.
- Practical implementation of geospatial concepts.
- Secure authentication and role-based access.

19. Future Enhancements

- Online payment gateway integration.
- Live delivery tracking.
- Customer reviews and restaurant ratings.
- Coupons and discount management.
- Push, email, or SMS notifications.
- AI-based food recommendations.
- Estimated delivery time.

- Delivery route optimization.
- Restaurant analytics and reports.

20. Learning Outcomes

The project provides practical experience in MERN stack development, React component design, REST API development, MongoDB operations, authentication, authorization, geolocation, frontend-backend integration, debugging, testing, Git, and GitHub project management.

21. Conclusion

FoodieConnect is a practical full-stack food delivery application connecting customers, restaurants, delivery agents, and administrators. Its key feature is address-based location detection and restaurant matching, allowing users to enter a normal delivery address while geographical coordinates are handled in the background. The project demonstrates how modern web technologies can be combined to build a structured and scalable food delivery platform.

Project: FoodieConnect

Developer: Swati Muttannavar